



BITS Pilani
Pilani Campus

API-driven Cloud Native Solutions

Dr. Shreyas Rao
Associate Prof. (Off Campus), CSIS, BITS-Pilani



CC ZG506 / SE ZG506

Lecture No. 8

Contact Session wise Coverage

Session	Coverage	
CS01	API Basics (OpenAPI, REST, gRPC, GraphQL)	Basics
CS02	API Basics + Cloud Native Application	
CS03	Cloud Native Application	
CS04	Introduction to Data Science & Machine Learning, DataOps & MLOps	Data Science & ML
CS05	API driven Cloud-native Data Pipeline	
CS06	API-driven ML pipelines	
CS07	Tools review; Case Study	
CS08	Basic APIs for AI Applications - NLP, CV, Speech recognition	Cognitive AI
CS09	Language Models (LLMs)	
CS10	RAG Models	
CS11	RAG Models	
CS12	Cloud Native Application Deployment	IoT
CS13	IoT and Data Analytics	
CS14	APIs for IoT and Data Analytics	
CS15	Serverless Functions; Case study	
CS16	Course review and discussion	

Agenda

AI and Cognitive Services

Part-A: Basic APIs for AI applications

- Hugging Face APIs
- Natural Language Processing APIs
- Computer Vision APIs
- Speech Recognition APIs
- Case Study: API usage in AI Applications

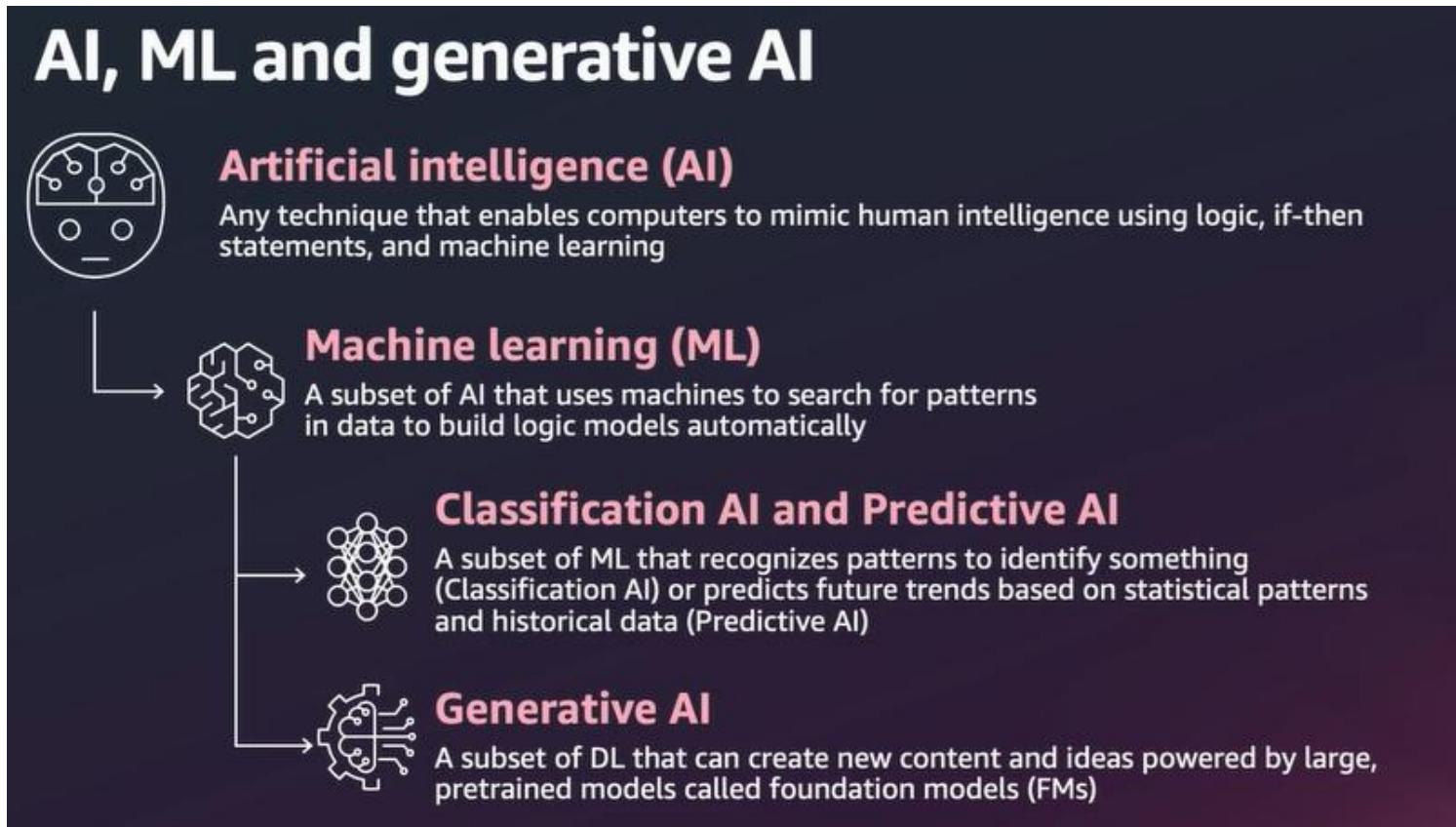


BITS Pilani
Pilani Campus



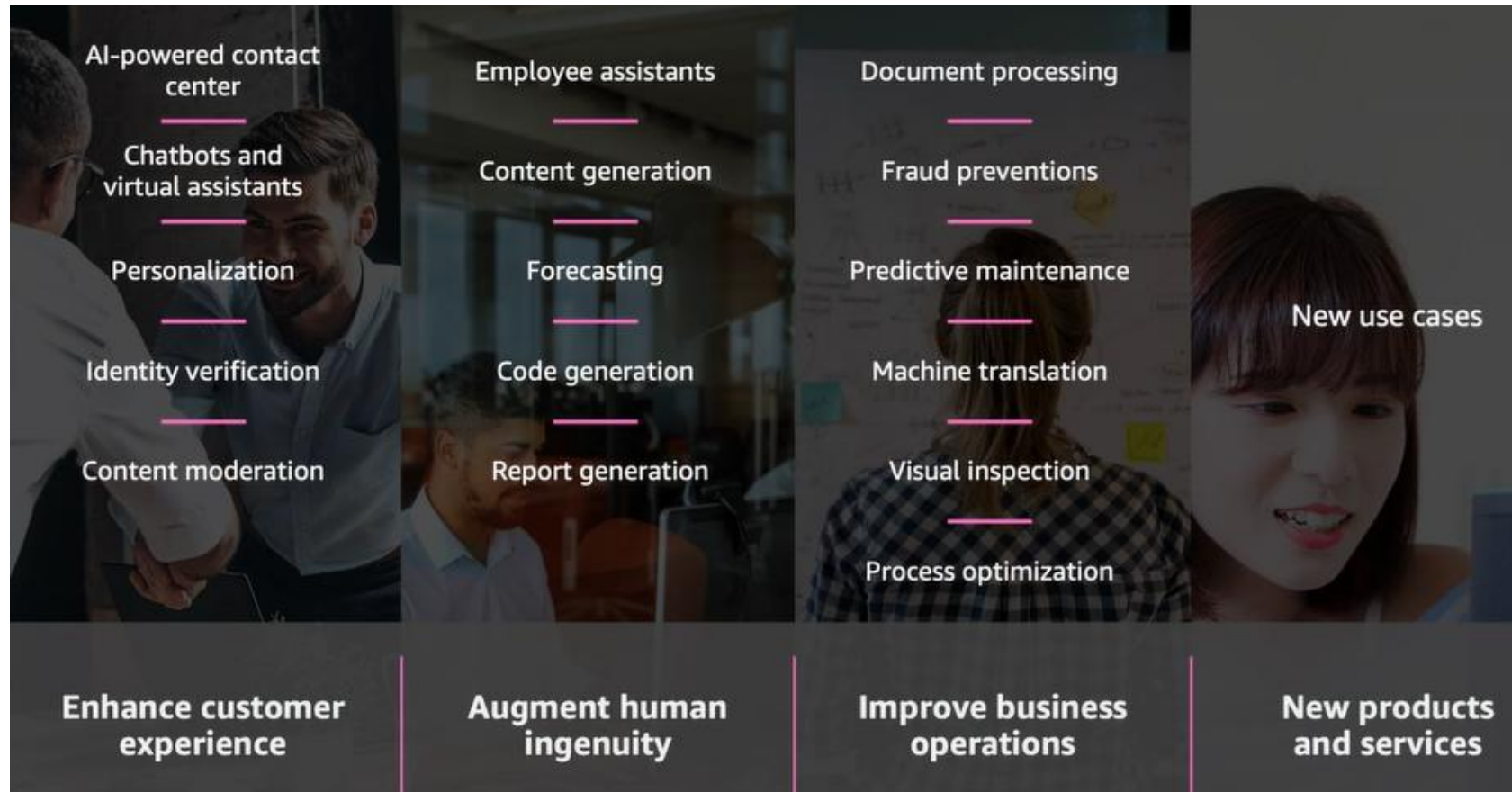
AI and Cognitive Services

AI, ML and Generative AI



Ref: Adapted from 'AWS Innovate Conference, 2024', Presentation by Melwin Pais, Senior Solution Architect, AWS

AI Use Cases



Ref: Adapted from 'AWS Innovate Conference, 2024', Presentation by Melwin Pais, Senior Solution Architect, AWS

Three Generations of Machine Learning

Gen 1: Basic Machine Learning

Effort: Create Labeled datasets, Train Models using basic ML algorithms, Measure Performance

Gen 2: Deep Learning [based on Artificial Neural Networks]

[Emergence of Big Data]

Effort: Re-Train Models using DL algorithms and measure performance

Gen 3: Transfer Learning / Transformers

[Pre-trained models / Fine-tuned models available]

Effort: Use APIs in various applications on a pay-per-use basis. No model training required

Pre-Trained vs Fine-Tuned Models

Training data

Pre-trained models are trained on large datasets, while fine-tuned models are trained on specific examples.

Purpose

Pre-trained models are general-purpose base models, while fine-tuned models are adapted for specific tasks or domains.

Efficiency

Pre-training is computationally intensive, while fine-tuning is more efficient because it builds on the pre-trained model.

Pre-Trained vs Fine-Tuned Models

Performance

Fine-tuning can improve a model's performance on specific tasks, and can often outperform models trained from scratch.

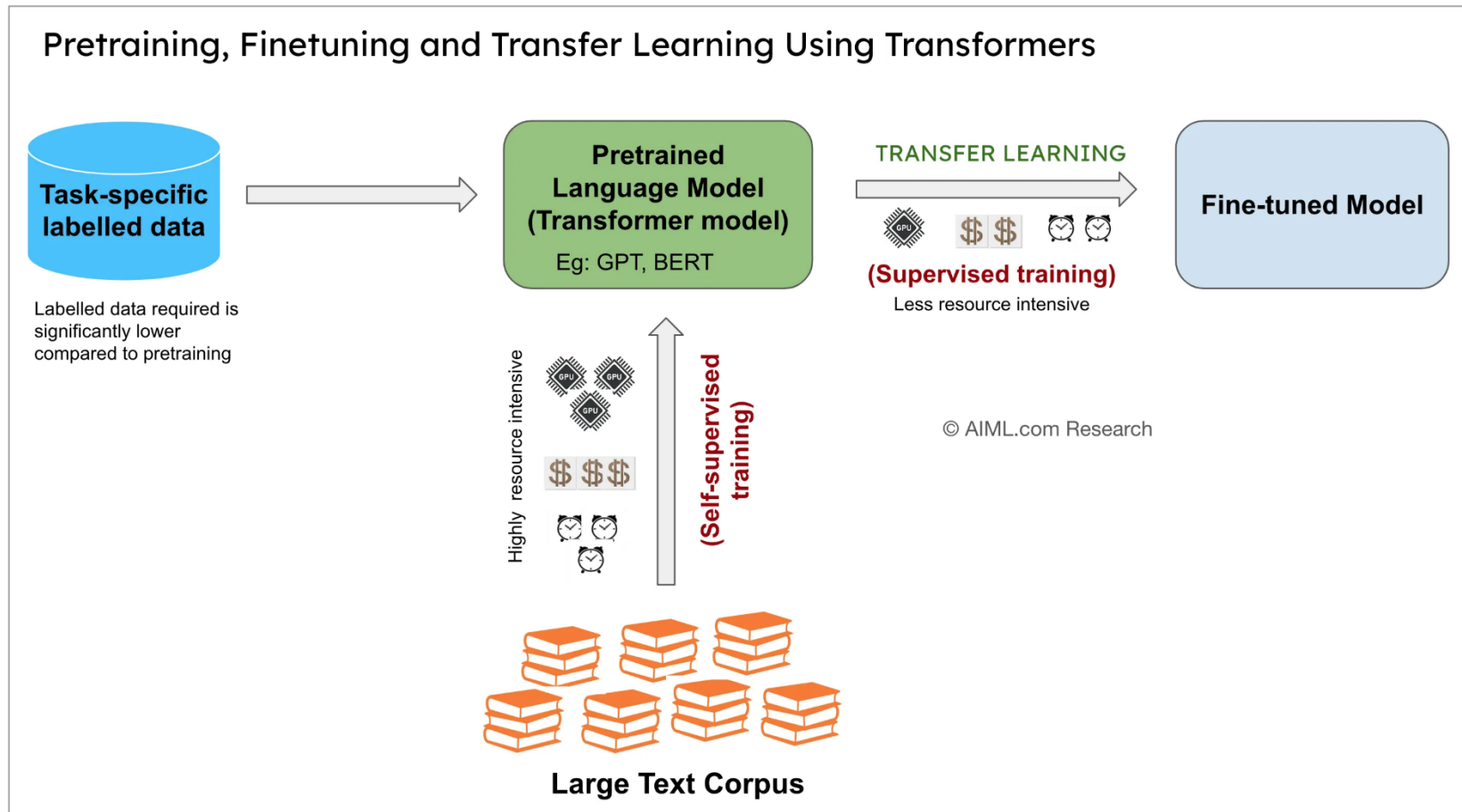
Transfer learning

Fine-tuning allows models to apply knowledge gained from one task to another, potentially unrelated task.

Interpretability

Fine-tuning can make models more interpretable, which can help with debugging and understanding the model's behavior.

Pre-Trained vs Fine-Tuned Models



AI-based Cognitive Services

Cognition - “Mental action or process of acquiring knowledge and understanding through thought, experience, and the senses”

- Cloud-based cognitive services are a set of AI-powered APIs and tools provided by cloud platforms (such as Microsoft Azure, AWS, or Google Cloud) that enable developers to easily integrate AI functionalities into their applications without needing in-depth knowledge of AI or machine learning.
- These services use cloud infrastructure to offer scalable, ready-to-use models and algorithms for tasks involving human-like cognitive abilities.



Cognitive Services

Ex: Azure AI Services

- Set of cloud-based APIs that you can use in AI applications and data flows.
- It provides pretrained models that are ready to use in your applications, requiring no data and no model training on your part.
- The services are developed by the Microsoft AI and Research team and expose the latest deep learning algorithms.
- They're consumed over HTTP REST interfaces.
- In addition, SDKs are available for many common application development frameworks.

<https://learn.microsoft.com/en-us/azure/architecture/data-guide/technology-choices/cognitive-services>



Benefits

- Minimal development effort for state-of-the-art AI services.
- Use predefined algorithms or create custom algorithms on top of pre-built libraries.
- Easy integration into apps via HTTP REST interfaces.
- Developers and data scientists of all skill levels can easily add AI capabilities to apps.

<https://learn.microsoft.com/en-us/azure/architecture/data-guide/technology-choices/cognitive-services>

Types of Services

Service	Description
Language	Services that provide Natural Language Processing (NLP) features for understanding and analyzing text.
Speech	Services that provide speech capabilities like speech-to-text, text-to-speech, speech translation, and speaker recognition
Computer Vision	Services that provide image and video recognition capabilities
Decision Services	Services that provide NLP features to produce recommendations for informed and efficient decision-making
Azure OpenAI Service	Service provides REST API access to powerful OpenAI language models.

<https://learn.microsoft.com/en-us/azure/architecture/data-guide/technology-choices/cognitive-services>



HuggingFace

Kaggle vs HuggingFace

- Kaggle is a popular dataset store for Generation 1 ML and Generation 2 DL
- URL - <https://www.kaggle.com/>
- HuggingFace is a popular dataset and model store for Generation 3 Transformers
- URL - <https://huggingface.co/>



HuggingFace

- HuggingFace Hub is a open source platform for hosting:
 - Git-based code repositories, including discussions and pull requests for projects.
 - Models, also with Git-based version control
 - Datasets, mainly in text, images, and audio



HuggingFace APIs

- Hugging Face APIs are cloud-based cognitive services, particularly in the areas of NLP, CV and Speech recognition.
- Hugging Face provides a variety of pre-trained models and APIs that developers can use to build intelligent applications without needing to train models from scratch [For Inferencing purposes]
- Website - <https://huggingface.co/>



HuggingFace Key Terminologies

Task

- Tasks refer to the specific machine learning or AI problems that you want to solve using Hugging Face models.
- Each task represents a different type of problem, often tied to a particular application of machine learning or NLP.

Common tasks in Hugging Face:

- **Text Classification:** Categorizing text into predefined categories (e.g., sentiment analysis, spam detection).
- **Named Entity Recognition (NER):** Identifying and classifying entities (such as people, locations, organizations) in text.

Task

Common Tasks

- **Text Generation:** Generating text, such as auto-completing or generating a paragraph from a prompt (e.g., using GPT models).
- **Machine Translation:** Translating text from one language to another.
- **Question Answering:** Answering a question based on a given passage or context (e.g., SQuAD-style tasks).
- **Summarization:** Creating a shorter version of a longer text while preserving the core meaning.
- **Image Classification:** Classifying images into predefined categories (using models trained on images).
- **Speech Recognition:** Converting spoken language into written text.
- **Object Detection:** Identifying objects within an image.

Hugging Face provides an extensive list of pre-built models tailored to each of these tasks.

Models

- Models in Hugging Face refer to the machine learning models that are pre-trained to solve specific tasks.

These models can be either:

1. **Pre-trained:** Already trained on large datasets for common tasks (e.g., language modeling, text classification, translation).
2. **Fine-tuned:** Adapted from a pre-trained model to a more specific task using a smaller, domain-specific dataset.



Models

Common Model Types

- **Transformer-based Models:** Hugging Face is known for its support of transformer architectures, which excel in NLP tasks.

Popular transformer models include:

- **BERT** (Bidirectional Encoder Representations from Transformers): Used for tasks like text classification, NER, and question answering.
- **GPT** (Generative Pre-trained Transformer): Used for text generation and completion.
- **T5** (Text-To-Text Transfer Transformer): A versatile model for a variety of text tasks by framing them as a text-to-text problem (e.g., translation, summarization).
- **DistilBERT**: A smaller, faster version of BERT with reduced computational requirements.
- **RoBERTa**, **XLNet**, **ALBERT**: Variants of BERT optimized for different tasks or efficiencies.

Dataset

- Datasets in Hugging Face are collections of data used to train, fine-tune, or evaluate models.
- Hugging Face provides an extensive hub of datasets that cover a variety of tasks across different domains.
- These datasets can be accessed through the Hugging Face Datasets library.



Features of Dataset

Pre-defined datasets

- SQuAD (Stanford Question Answering Dataset): A dataset for question-answering tasks.
- GLUE (General Language Understanding Evaluation): A collection of datasets for evaluating NLP models on a wide range of tasks like text classification, sentence similarity, etc.
- IMDB: A dataset used for sentiment analysis of movie reviews.
- COCO (Common Objects in Context): An image dataset used for image classification and object detection tasks.
- LibriSpeech: A dataset for speech recognition tasks.

Custom datasets

- You can upload and share your own datasets for use in fine-tuning models



How Tasks, Models, and Datasets interact

- Tasks define what you want to achieve, such as text classification or translation.
- Models are the pre-trained or fine-tuned algorithms that perform these tasks.
- Datasets provide the necessary training or evaluation data for the models to learn from or be tested against.

Example Workflow

1. **Task Selection:** Choose a task, e.g., sentiment analysis.
2. **Model Selection:** Pick a pre-trained model like BERT fine-tuned for sentiment analysis from the Hugging Face Hub.
3. **Dataset Selection:** Use a dataset like the IMDB movie reviews dataset to fine-tune or test your model. [Optional]
4. **Training or Inference:** Train the model on the selected dataset [[Fine-Tune](#)] or use it to make predictions on new data [[Inference](#)].



HuggingFace Model Card

- A model card in Hugging Face is a documentation tool that provides important information about a specific machine learning model hosted on the Hugging Face Hub.
- Model cards are designed to offer transparency about a model's performance, intended use cases, limitations, and the data it was trained on, helping developers, researchers, and users understand how to effectively and responsibly use the model.



Sample HuggingFace Model Card

Model Name: “nlptown/bert-base-multilingual-uncased-sentiment”

Description: This is a bert-base-multilingual-uncased model fine-tuned for **sentiment analysis** on product reviews in six languages: English, Dutch, German, French, Spanish, and Italian.

Intended Use: It predicts the sentiment of the review as a number of stars (between 1 and 5), where 1 stands for most negative sentiment, and 5 for most positive sentiment.

Training Data: The model was fine-tuned on multiple datasets in English, Dutch, German etc.

Try it out in the Inference Provider!

Question!

How do you select the appropriate Model from HuggingFace? Since so many tasks, models, and datasets are present?

Answer

There is NO SINGLE solution here

1. Select the kind of problem : General purpose or specific
2. Select the task: NLP, Computer Vision or Speech Recognition etc.
3. Select the sub-task: Text generation or object identification or question answering or language translation etc.
4. Consider type of Language Model required: Large, Small etc.
5. Consider Performance metrics (Accuracy, F1) of the model if available, based on the sub-task
6. Test the dataset on various models before final selection. Use “Inference Providers for quick assessment!”

Some Models and Tasks - AWS

Meta AI	AI21 labs	co:here		stability ai	Lightn	alexia
Models Llama 3 Guard 7B, 8B, 70B Code Llama 7B, 13B, 34B	Models Jurassic-2 Ultra, Mid Jamba Instruct	Models Command Command Light	Models Falcon-7B, 40B, 180B Mistral 7B Open LLaMA RedPajama MPT-7B, Dolly BloomZ 176B Flan T-5 models (8 variants) DistilGPT2 GPT NeoXT Bloom models	Models Stable Diffusion XL 1.0 2.1 base Upscaling Inpainting	Models Lyra-Fr 10B, Mini	Models AlexaTM 20B
Tasks Question answering Chat Summarization Paraphrasing Sentiment analysis Text generation	Tasks Text generation Long-form generation Summarization Paraphrasing Chat Information extraction Question answering Classification	Tasks Text generation Information extraction Question answering Summarization	Tasks Machine translation Question answering Summarization Annotation Data generation	Tasks Generate photo-realistic images from text input Improve quality of generated images	Tasks Text generation Keyword extraction Information extraction Question answering Summarization Sentiment analysis Classification	Tasks Machine translation Question answering Summarization Annotation Data generation
				Features Fine-tuning on Stable Diffusion 2.1 base model		

Ref: Adapted from 'AWS Innovate Conference, 2024', Presentation by Melwin Pais, Senior Solution Architect, AWS

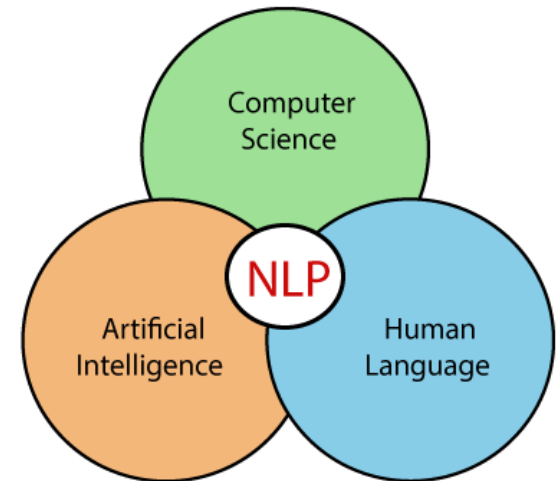


BITS Pilani
Pilani Campus



Basics of Natural Language Processing

- NLP stands for **Natural Language Processing**
- Subfield of Computer Science, Human language (Linguistics), and Artificial Intelligence
- It is the technology that is used by machines to understand, analyze, manipulate, and interpret human's languages.



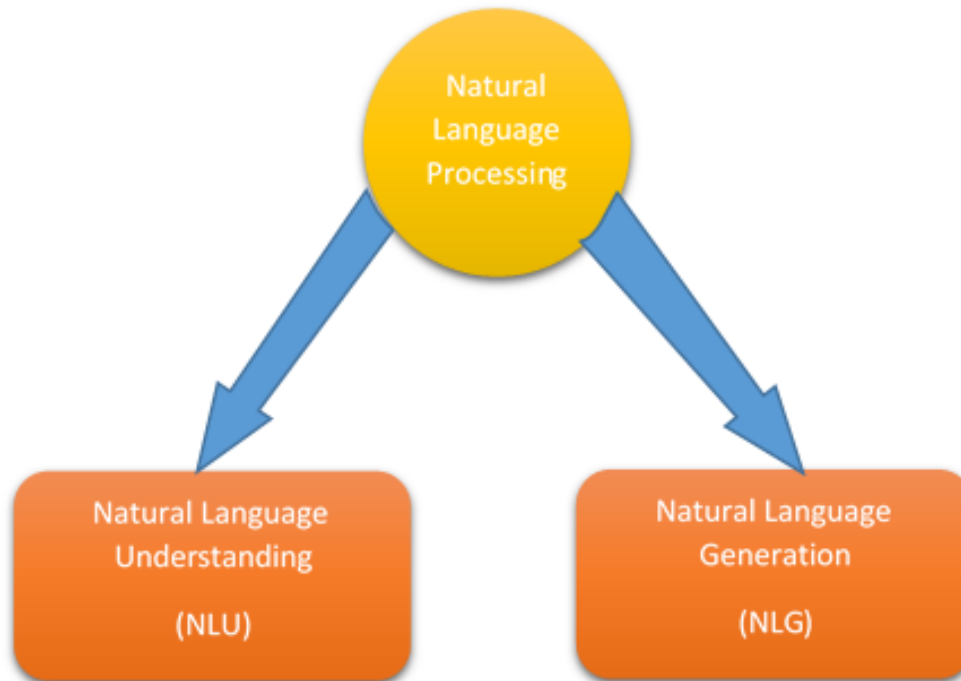
- Users can provide two kinds of input to Machine → **Text** or **Speech**
- Machine has to first **understand the human speech** in order to act on it

Examples:

- ❑ **User types a message “BITS Pilani” in Google Search**
- ❑ **User tells Alexa “Play me Kishore Kumar Songs”**

In both the examples, Machine should understand the **Syntax, Semantics, Context and Intent** of the statement, in order to provide a favorable reply.

Components of NLP



**Understands the
Human Speech**

**Process and
Action**

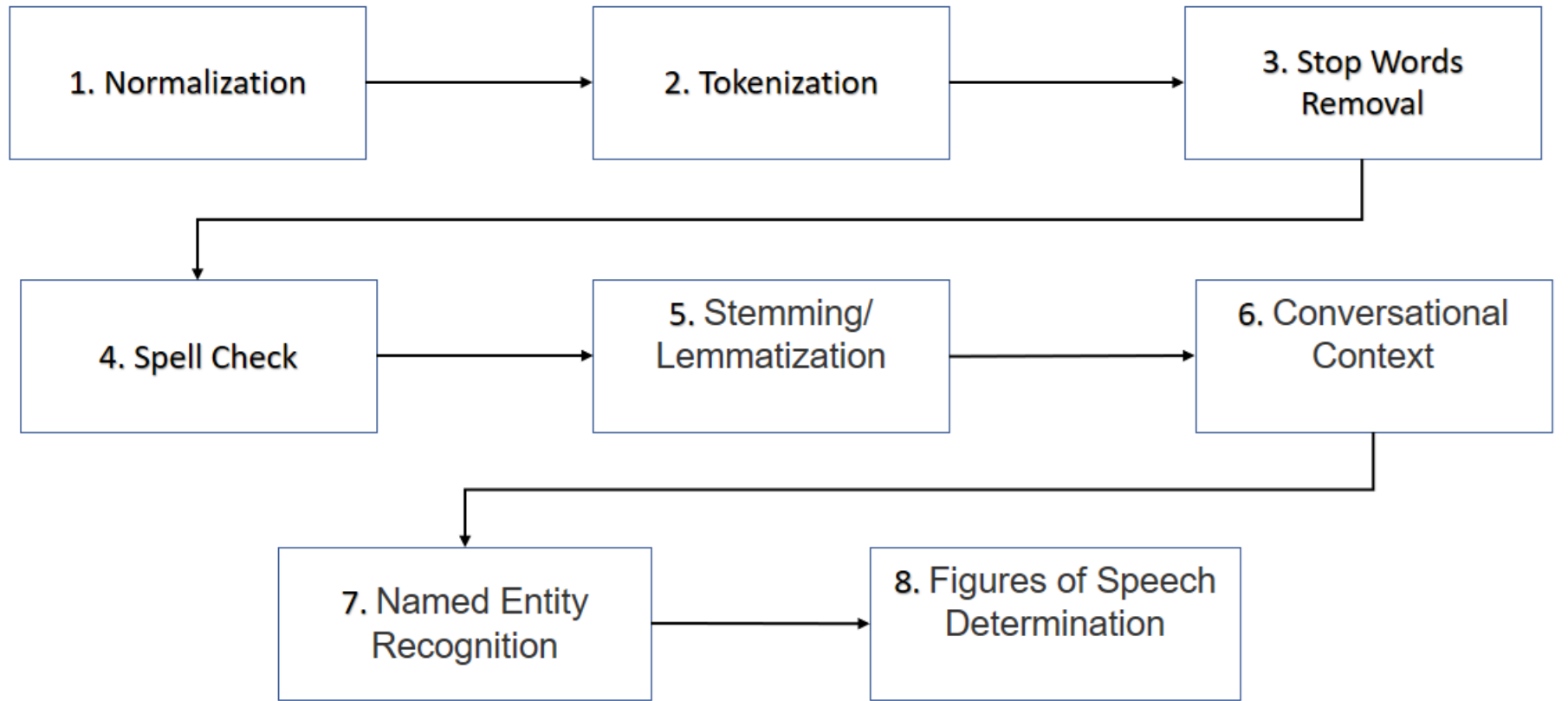
NLU

- Understand the Text
- Understand Syntax, Context, Sentiment, Semantics and Intent from the text
- Syntactic Analysis
 - Lemmatization, Stemming, Word Segmentation, POS tagging etc.
- Semantic Analysis
 - Named Entity Recognition (NER), Word Sense Disambiguation etc.

NLG

- Produce meaningful sentences in human-understandable text
- NLG Models:
 - Markov Chain
 - Recurrent Neural Network (RNN)
 - Long Short-Term Memory (LSTM)
 - Transformers

Sample NLP Pipeline for NLU



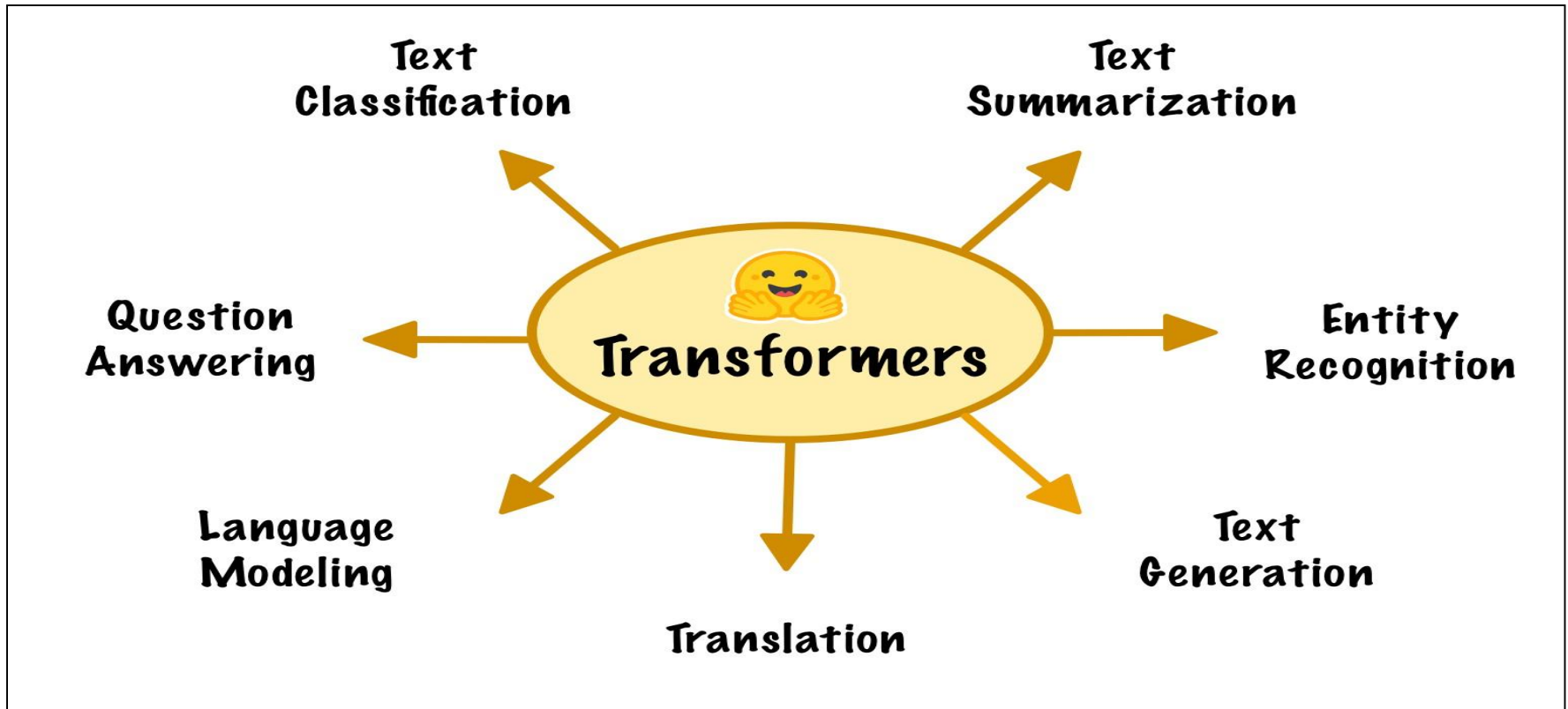
- Stanford CoreNLP Parser (Python)
- NLTK – Natural Language Toolkit (Python)
- SpaCy (Python)
- Apache OpenNLP (opensource Java Library)

Sample Python Program to demonstrate NLU and NLP Pipeline
using NLTK Library

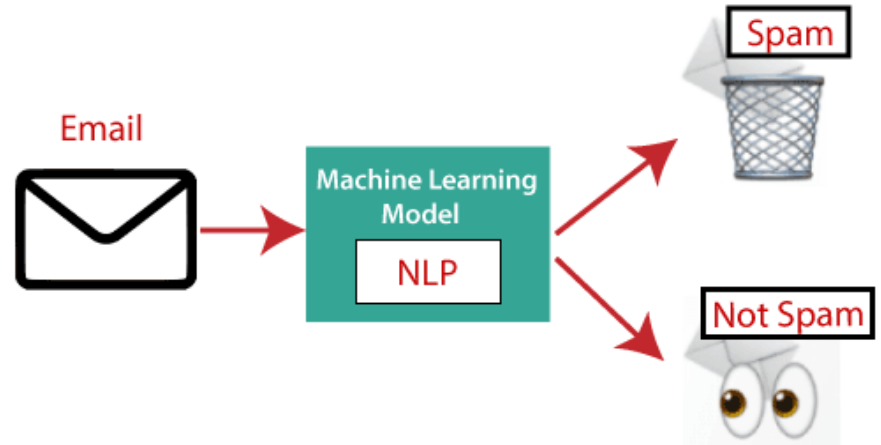


Application of Transformers in NLP

Applications of Transformers in NLP



Applications of Transformers (NLP)



Question – Answering

Question Answering focuses on building systems that automatically answer the questions asked by humans in a natural language.

Ex: Virtual Assistants (Alexa, Google Mini)

Spam detection

Is used to detect unwanted e-mails getting to a user's inbox.

Applications of Transformers (NLP)



Sentiment Analysis

Also called “Opinion Mining”

This application is implemented through a combination of NLP (Natural Language Processing) and statistics by assigning the values to the text (positive, negative, or neutral), identify the mood or emotion of the context (happy, sad, angry, etc.)

Applications of Transformers (NLP)

English

↔

Kannada

dhriti bot
provides mental
health support
during covid

×

ಧೃತಿ ಬೋಟ್ ಕೋವಿಡ್
ಸಮಯದಲ್ಲಿ ಮಾನಸಿಕ
ಆರೋಗ್ಯ ಬೆಂಬಲವನ್ನು
ಒದಗಿಸುತ್ತದೆ
Dhṛti bōṭ kōviḍ samayadalli mānasika
ārōgya bembalavannu odagisuttade

Machine translation

Is used to translate text or speech from one natural language to another natural language.

Applications of Transformers (NLP)

Artificial Intelligence is the next big technology trend in computing that will be used in future. Today, the AI technology is only used for solving some problems. In the near future, we can use this technology for solving health related problems and for improving the efficiency of our lives.

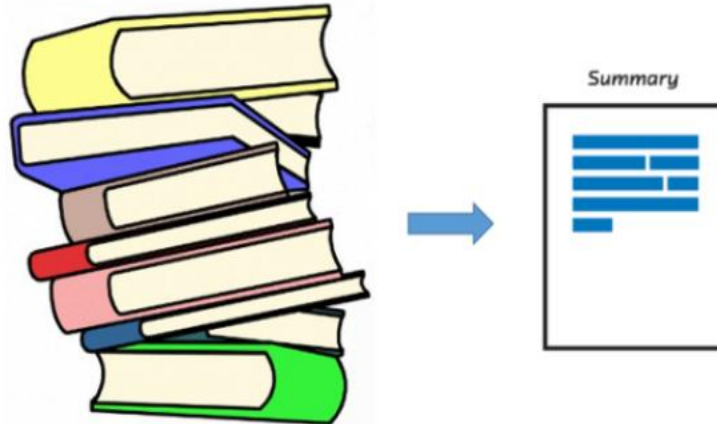
Written by Transformer · transformer.huggingface.co 

Text Generation

Used to generate text automatically based on user data and context.

Ex: ChatGPT

Applications of Transformers (NLP)



Text Summarization / Text Paraphrasing / GEC

Text Summarizer tools: Summarize Bot,
Resoomer, SMMRY

Text Paraphrasing tools – Quillbot, Spinbot,
Grammarly, GoParaphrase etc.

GEC (Grammatical Error Checker) - Grammarly



BITS Pilani
Pilani Campus



Practical NLP with Transformers



NLP Task 1 -> Sentiment Analysis

[Comes under “Task Classification” category in HuggingFace]

Usage Example

```
from transformers import pipeline  
  
classifier = pipeline('sentiment-analysis', model='distilbert-base-uncased-finetuned-sst-2-english')  
  
result = classifier("I love this movie!")  
  
print(result)
```

Here “Pipeline” is the high level API

Try it out in Google Colab!

Change “Runtime Type” to “T4 GPU” if required

<https://colab.research.google.com/drive/1vc7w2gNHld9CHkrSOmTaluPrNRxzjDY0?usp=sharing>



NLP Task 2 -> Language Translation

Reference - <https://huggingface.co/Helsinki-NLP/opus-mt-en-hi>

[Read the Model Card; Go to -> “Use this Model -> Transformers” and view code]

Try it out in Google Colab!

<https://colab.research.google.com/drive/1vc7w2gNHld9CHkrSOmTaluPrNRxzjDY0?usp=sharing>



NLP Task 3 -> Question Answering

Reference - <https://huggingface.co/distilbert/distilbert-base-uncased-distilled-squad>

Try it out in Google Colab!

<https://colab.research.google.com/drive/1vc7w2gNHld9CHkrSOmTaluPrNRxzjDY0?usp=sharing>



Computer Vision APIs

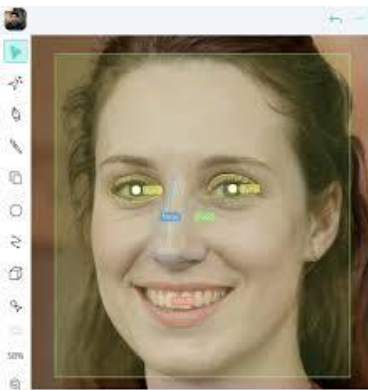
Computer Vision

Computer vision is a field that uses algorithms to **process images** and **videos** to perform tasks such as



Object detection

Locating and identifying objects in an image or video. This is used in many applications, including surveillance, robotics, image and video analysis.



Face recognition

A widely used application of computer vision, face recognition is used to unlock devices and identify security threats.

BITS Pilani, Pilani Campus

Computer Vision

Computer vision is a field that uses algorithms to **process images** and **videos** to perform tasks such as



Image restoration

An important task in computer vision, image restoration can include image inpainting, image deblurring, pixel interpolation, and image denoising

Object Tracking is a continuous CV task that aims to track a specific **object** or multiple objects over a sequence of frames

Scene reconstruction is the process of reconstructing a 3D digital version of a real-world object from pictures or scans of the object

Ex: Generating a **3D model** of a historical building using old photos

Colorization is adding realistic colours to black and white pictures.

CV Task -> Image Classification

Model - "facebook/deit-base-distilled-patch16-224"

Code

```
from transformers import pipeline
# Load the image classification pipeline
image_classifier = pipeline("image-classification", model="facebook/deit-base-distilled-patch16-224")
# Use a new image URL (make sure it's a valid and accessible image URL)
url = "https://upload.wikimedia.org/wikipedia/commons/9/9a/Pug_600.jpg"
# Perform image classification using the URL directly
result = image_classifier(url)
# Print the result
print(result)
```

<https://colab.research.google.com/drive/1vc7w2gNHld9CHkrSOmTaluPrNRxzjDY0#scrollTo=m dMmulTsRHnO>



BITS Pilani
Pilani Campus



Speech Recognition APIs

Automatic Speech Recognition (ASR)



- Is a technology that converts spoken language into text.
- It's the foundational component in speech recognition systems, enabling computers and devices to "understand" and process human speech in real-time or through recorded audio.
- ASR is widely used in applications like virtual assistants (e.g., Siri, Google Assistant, Alexa), transcription services, call centers, and more.

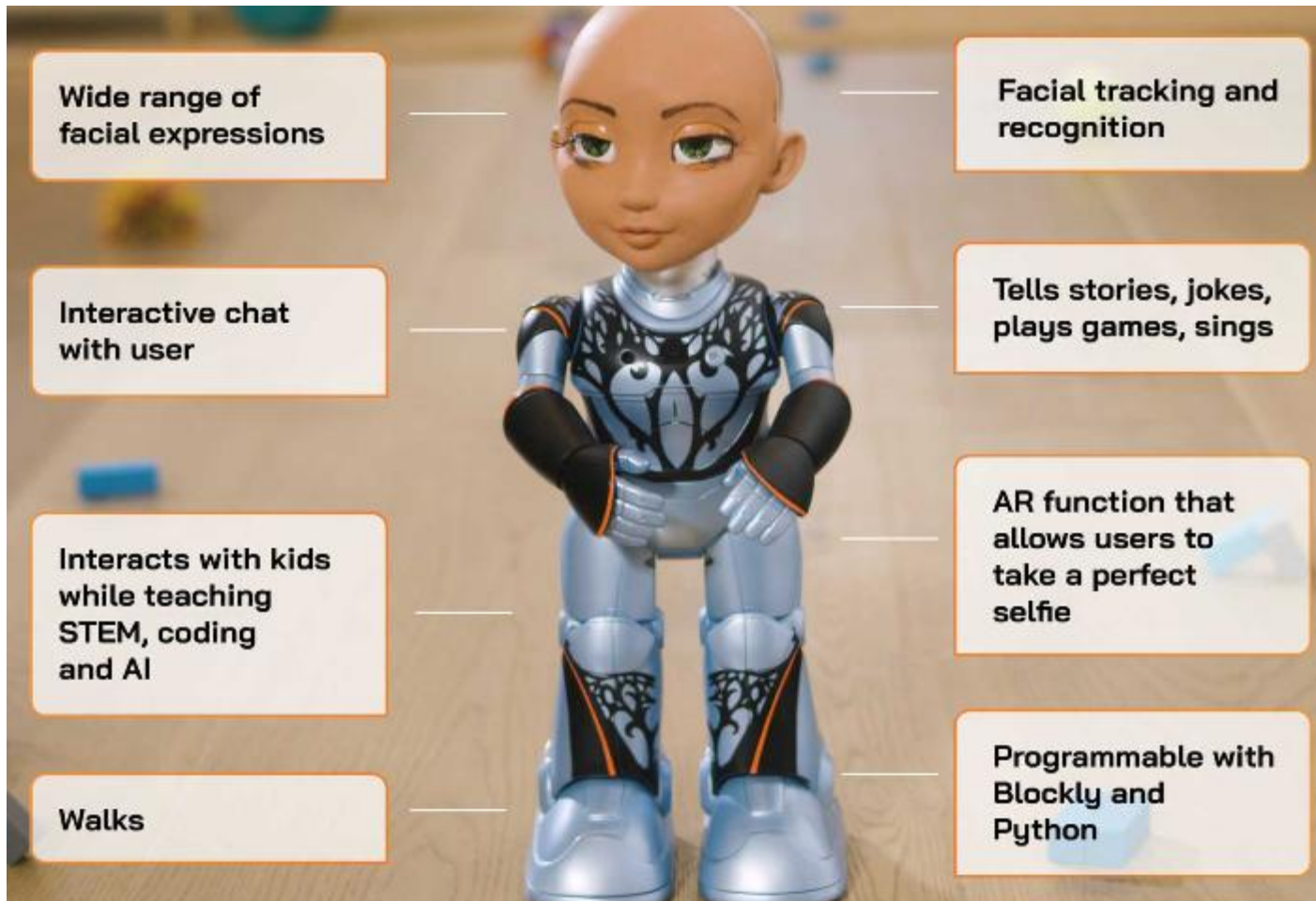
Key Features of ASR



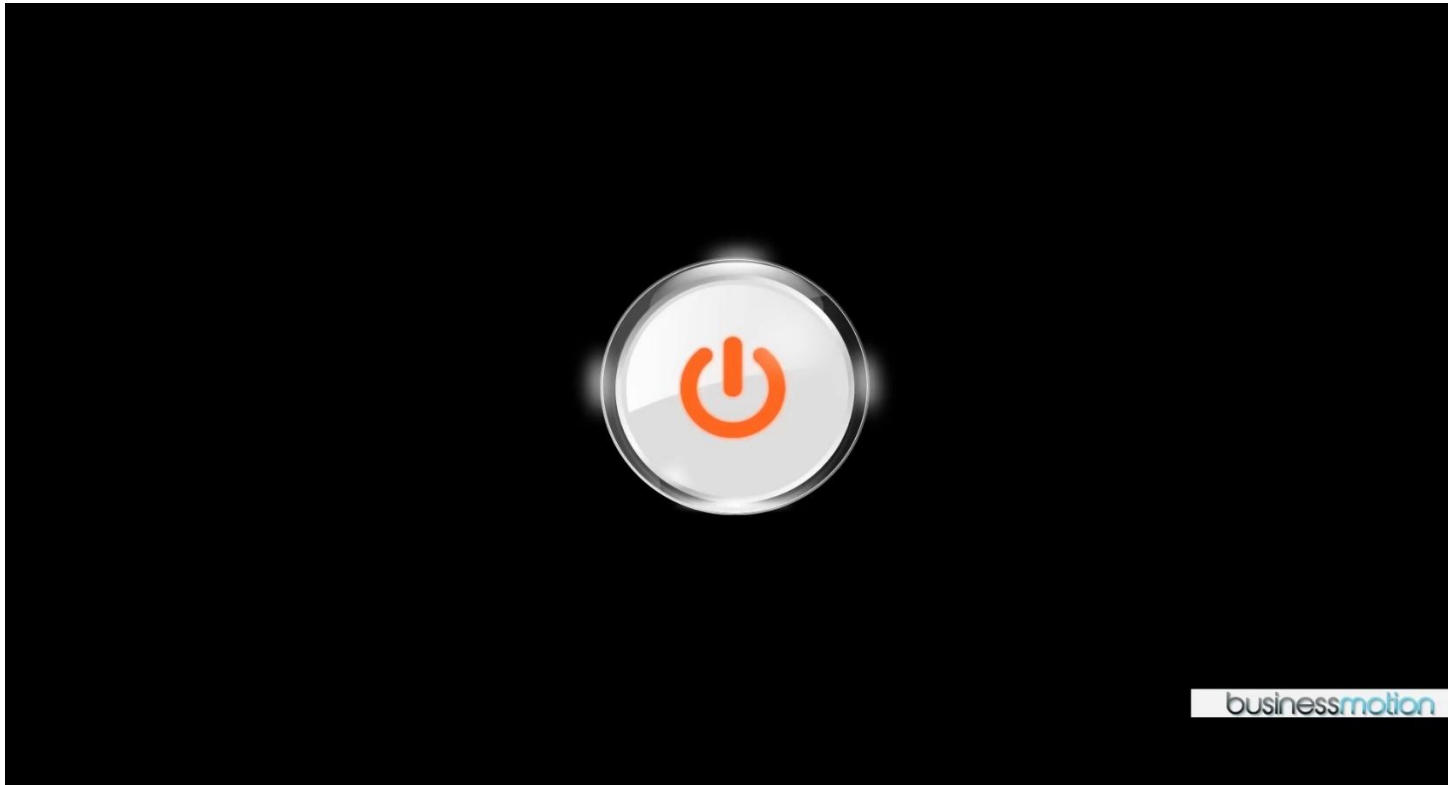
- **Feature Extraction:** The system captures audio features, such as phonemes (small sound units), tone, pitch, and frequency.
- **Acoustic Modeling:** This step helps the system recognize various sound patterns and phonetic elements in speech.
- **Language Modeling:** The ASR system applies linguistic rules, understanding grammar, context, and vocabulary, to improve accuracy.
- **Decoding:** The system processes the audio input to generate the most likely text output based on the sounds and language patterns.

Modern ASR systems use AI models, especially deep learning, to improve recognition accuracy and handle nuances like accents, background noise, and varying speech speeds.

Case Study: Sophia Robot by Hanson Robotics



Sophia Robot by Hanson Robotics



<https://www.youtube.com/watch?v=G-zyTIZQYpE&t=316s>

Watch from 4.30 minutes till the end of the video



Thank You!